Case Study 94: Secure Online Voting System

Pratik Kumar Swain

Problem Statement

1

Manual Vulnerabilities

Traditional paper-based systems are slow and heavily prone to human error during the time-consuming counting phase, leading to potential inaccuracies.

2

Security Risks

Without automated tracking mechanisms, preventing double-voting or unauthorized participation remains a significant challenge in manual setups.

3

Lack of Control

Monitoring critical election timelines such as opening and closing polls is often disorganized and difficult to enforce in manual voting environments.

4

Delayed Results

Tallying votes manually takes hours or even days, whereas a digital system offers instant results and immediate transparency.

The Solution

1

OOP Implementation

The system uses distinct Voter and Candidate classes to encapsulate sensitive data and maintain clear separation of concerns.

2

Security Logic

A built-in hasVoted flag within the Voter class ensures every registered user can only cast a single ballot, preventing duplicate votes.

3

Interactive Control

The Election class manages the live state of the polls using a boolean isRunning flag, preventing pre-election or post-election tampering.

4

Automated Tallying

The code automatically iterates through candidate data to determine the winner based on the highest voteCount, eliminating human error.

Working Method

The system follows a linear, secure pipeline from registration to final results:

Registration

Admin adds unique Voter IDs and Candidate profiles to the system vectors for secure tracking.

Activation

The election is toggled to "Live" state using the startElection() method, opening polls for voting.

Authentication

Voters must provide a matching ID and Password to access the ballot and verify their identity.

The Vote

Users select a candidate ID and must provide a final 'y/n' confirmation to record their choice permanently.

State Update

The system increments the candidate's vote count and permanently flips the voter's hasVoted status to true, preventing duplicates.

Conclusion

1

Data Integrity

By using C++ Standard Template Library (std::vector), we ensure data is managed efficiently during the session without memory leaks.

2

Zero Error Margin

Automated result calculation removes the bias and inaccuracies of manual counting, ensuring 100% accuracy in vote tallies.

3

Scalability

While currently a console prototype, the logic provides a solid blueprint for encrypted, network-based voting platforms with millions of users.

4

User-Friendly

A simple menu-driven interface makes the process accessible even for non-technical users without compromising security.

ITM SKILLS UNIVERSITY

Thank You